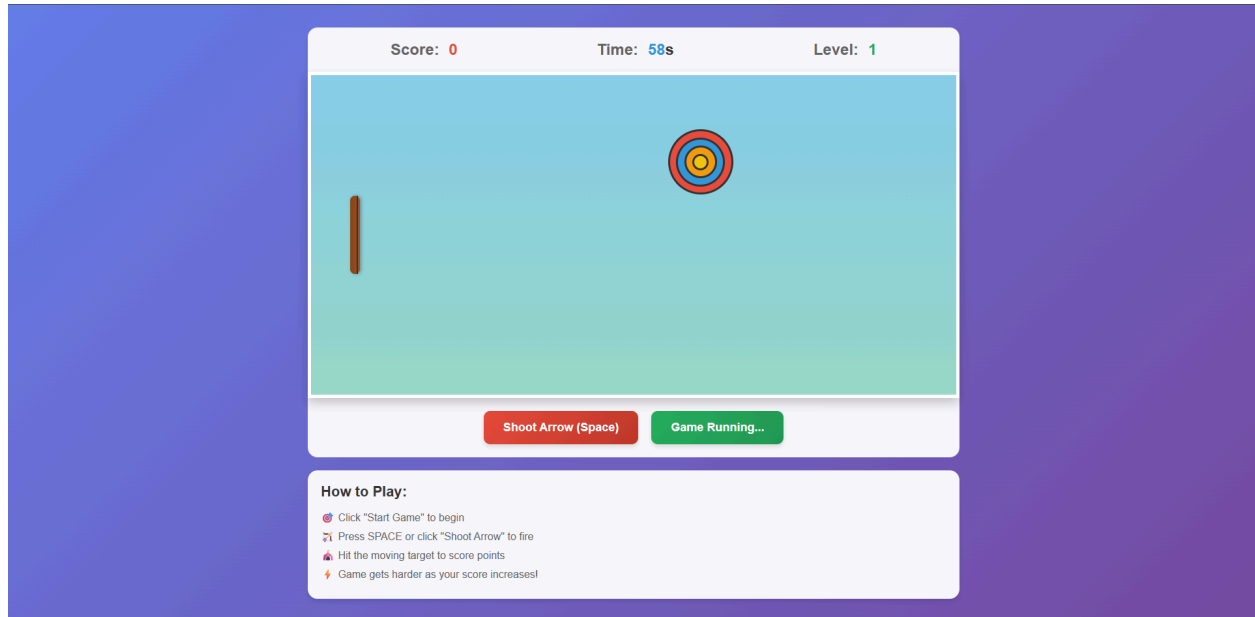


Analysis

The game made by AI is not at all what it was supposed to look like:



The bow isn't even a bow. The target moves randomly anywhere on screen instead of up and down on a fixed axis. The arrow shoots itself towards the target instead of shooting where the user's cursor is. The background is nothing but a gradient. Arrow speed changes at random times, sometimes it shoots very fast while other times its very slow. Overall, very bad game made by AI

Prompt

[attached image of what game should look like]

This archery game is developed using HTML, CSS, and JavaScript where all game elements such as the bow, arrow, and target are created using div elements. The player shoots an arrow by clicking or pressing a control, and the arrow moves towards the target using JavaScript logic. The target is not stationary; it moves continuously within a defined area, increasing the difficulty of the game. When the arrow hits the moving target, the score increases. The game uses JavaScript event handling and DOM manipulation to control movement, collision detection, and score updates.

Criteria

Updated Description (with coding changes)

Marks

DOM Element & Object Usage

Correctly selects DOM elements using modern JavaScript methods (querySelector, getElementById). Uses variables/constants appropriately and avoids hard-coding values. Elements are created or updated dynamically where needed.

Event Handling Logic

Implements event listeners efficiently using addEventListener. Correctly distinguishes between target clicks and background clicks using event objects. Prevents unintended behavior such as event bubbling when required.

Function & Code Structure

Code is modular and readable with well-named functions for movement, timer, score update, and difficulty control. Uses arrow functions or standard functions consistently and avoids duplicated logic.

You create separate functions for different tasks, for example:

moveTarget() → moves the archery target

startTimer() → starts and updates the countdown

updateScore() → increases score when the target is hit

increaseDifficulty() → makes the game harder

Function names clearly describe what they do.

You do not repeat the same code in multiple places.

You use one style of functions consistently:

Either arrow functions () => {}

Or normal functions function name() {}

Game Logic & State

Maintains clear game state variables (score, time, game status). Correctly handles timer countdown, game over condition, score updates, and reset/restart logic without bugs.

You use variables to track the game state, such as:

score → number of targets hit

timeLeft → remaining game time

gameOver → whether the game has ended

The timer counts down correctly every second.

When time reaches zero:

The game stops

Target movement stops

A game over message is shown

The score only increases when the player hits the target.

The game can be reset or restarted without errors.

Dynamic Styling & Difficulty

Dynamically updates styles using JavaScript (size, speed, color, position). Difficulty increases logically based on score or time (e.g., faster movement, smaller target). Changes are smooth and responsive.

Extra Help:

This archery game is developed using HTML, CSS, and JavaScript where all game elements such as the bow, arrow, and target are created using div elements. The player shoots an arrow by clicking or pressing a control, and the arrow moves towards the target using JavaScript logic. The target is not stationary; it moves continuously within a defined area, increasing the difficulty of the game. When the arrow hits the moving target, the score increases. The game uses JavaScript event handling and DOM manipulation to control movement, collision detection, and score updates.

make this exact game